

Replicated and Consistent Key-Value Storage in Go language

Adriano Trani

Department of Civil Engineering and Computer Engineering
University of Rome Tor Vergata
Rome, Italy
adriano.trani@students.uniroma2.eu

Abstract—The project consists in a very simple distributed, consistent and replicated key-value data memorization system. The system consists in a set of Docker containers deployed on an Amazon AWS EC2 instance. Interactions with the instance happens through a terminal. The project is composed of three different types of nodes: Client Proxy, responsible to handle the interactions between the system and the outside. Consensus Node, that manages the data in a distributed and consistent way using a simplified version of Raft algorithm. Snapshot Node, that creates snapshots for backup and restore purposes. The final analysis offers data about the performance of the implemented customized simplified Raft algorithm.

Keywords—Amazon AWS, Go Language, Key-Value Datastore, Performance Benchmarking, Docker Compose, PuTTY, Raft, gRPC and Protocol Buffers, Design Patterns, Docker Desktop.

I. INTRODUCTION

Maintaining data on a single device can lead to performance issues like the **Bottleneck Problem** or security issues like the **S.P.O.F. (Single Point of Failure)**.

This project provides a simple distributed and consistent **key-value data system** for memorizing data in a way more resilient against such problems using a set of replicated and cooperating nodes.

Such cooperation is mediated by a **simplified version of the Raft algorithm [1]** and the communication is handled by a custom **RPC-based** messaging system.

Already existing Key-Value datastores were not used in this system and they were substituted by self-made scripts in according to the provided specifications.

The same applies to open-source implementations of algorithms for Leader Election.

Amazon AWS [2] provided the virtual instances for the deployment of the system. Communication with such instances happened using **PuTTY**.

Docker Compose [3] handled the containerization of the application and **Docker Desktop** made possible to locally test the Go scripts on a Windows operating system.

gRPC and Protocol Buffers [4] handled every communication between nodes.

The implemented key-value datastore provides the following functionalities:

- **SEARCH** – **Search for a specific pair:** given a specific key, look up in the system if there is a corresponding pair key-value and return the value.
- **INSERT** – **Insert a new pair in the system:** given a pair key-value, add it to the system (if possible).

II. BACKGROUND AND THEORY

A. Raft Algorithm

Raft is a consensus algorithm used for managing a replicated log. Raft manages leader election, log replication and safety.

The idea behind Raft is quite simple: every node has a state that can be “leader”, “follower” or “candidate”; every node starts as a follower and wait for a heartbeat message from the leader. One of the of the leader duties is to periodically send a heartbeat message to every other node. Every node has an internal timer called “election timeout”. If a node receives a heartbeat before the end of the timer, then such timer will be restarted. If a timer reaches zero the node will assume the leader is crashed and it will switch to candidate state and will ask the other nodes to vote for itself using a particular type of message called “Request Vote”. Request Vote contains a specific number related to the node itself called “Raft term”. The value of the received term is one of the criteria used by the other nodes to decide to vote or not for the node that started the election. If a node receives most of the votes, it will become the new leader. Every node can vote only once per election round.

B. gRPC and Protocol Buffers

Using gRPC it is possible for a client application to call a method on a server application located in a different machine as if it were a local object.

By default, gRPC uses a mechanism called Protocol Buffers. The idea behind Protocol Buffers is to define messages and services used by gRPC in a proto file. This file is then used to automatically create code for implementing such services.

III. SOLUTION DESIGN

A. General Architecture

The project consists in a group of Docker containers and a set of scripts written in Go language.

Every container can assume only one of these roles:

- **Client Proxy:** bridge between the system and the outside world. It has the responsibility to receive commands from the outside and forward them to the leader node among the Consensus Nodes.
- **Consensus Node:** a group of nodes that manages the data and store them in a Docker Volume. It receives operations from the Proxy, it implements them and it sends an appropriate reply to the Proxy, if necessary. One of them is elected as a leader using a custom algorithm inspired to Raft in line with the provided specifics. All nodes work together as a datastore but only the leader receive instructions from the Client Proxy.
- **Snapshot Node:** a single node with the responsibility of gathering data from Consensus Nodes and use them to create snapshots for backup and restore purposes.

Different containers use different scripts; all scripts are written in Go language according to the provided specifics. More details in the following sections.

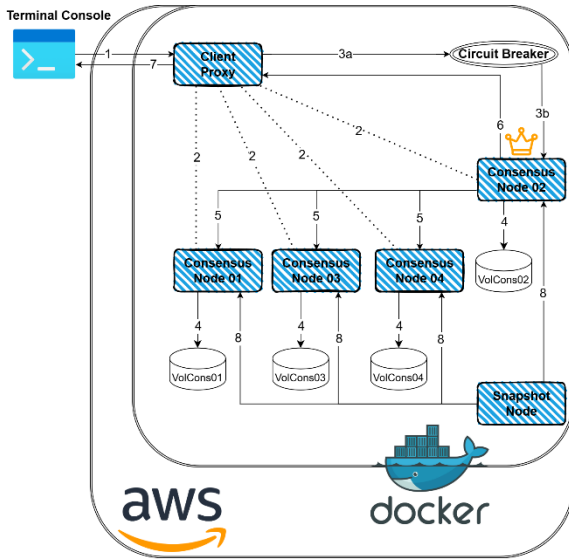


Fig. 1. Architecture of the system. The arrows with numbers indicate the ordered data flow.

B. Data flow

Reading and writing operations always follow this dataflow:

1. The command inserted in the terminal console is forwarded to the Client Proxy.
2. Client Proxy tries to discover who the leader node is and what are its hostname and port. The idea of using a centralized entity to store the information of who is the

leader was discarded because it's a practice that can lead to S.P.O.F. and bottleneck problems. Making the leader periodically contact the Client Proxy would be creating unnecessary traffic between them and would need the implementation of a heartbeat system on Client Proxy too. Given that every Consensus Node should always know who the leader is, the Client Proxy will contact one Consensus Node after the other until one of them provides the required information.

3. Client Proxy tries to contact the leader using the received address. This communication always passes through the circuit breaker.
 - i. If the dial operation with the provided node fails, then circuit breaker will be activated for a fixed amount of time.
 - ii. If the circuit breaker is already active, then the communication won't even begin.
 - iii. If there are no errors of sort, the leader node will receive the command.
4. The leader receives the command, and it appends it in its log. The leader also forwards the command to the other nodes.
5. Every container reads or writes its volume according to the provided command.
6. The answer to the command is sent from the leader back to the Client Proxy.
7. In any case the Client Proxy writes a suitable answer to the command in the terminal console.
8. Periodically the Snapshot Node asks log information to the Consensus Nodes.

C. Files organization

The project's functionalities are divided into several files (mainly Go files) according to their purpose:

1. **Docker-compose.yml** – Defines all the containers and the volumes used by the system. Also, it makes every container run its specific files and nothing else.
2. **Dockerfile** – Contains indispensable commands for environment configuration. Such commands make sure that:
 - gRPC and protobuf plugins are correctly installed.
 - Scripts written in Go language can be executed on the provided environment (both the EC2 instance and the local device).
 - Every file is copied in the working directory.
 - Go.mod file is generated (if not already present).
 - Protobuf files are generated starting from the provided sdcc.proto file.
 - Every Go dependence is present.
3. **proxyCode.go** – Contains the logic used to manage the communication between the system and the outside

world. There are scripts for reading and writing operations, scripts related to service discovery and a simple script that implements the “Circuit Breaker” pattern. Only the Client Proxy uses these scripts.

4. **conNodeCode.go** – Contains scripts related to the Consensus Nodes and used only by them. Several functions related to leader election and heartbeat messages can be found here.
5. **snapshotCode.go** – Contains scripts used by the Snapshot Node for communicating with the Consensus Nodes, for receiving their logs and for creating snapshots (for backup purpose) from them.
6. **commonComm.go** – Every script related to RPC communication between different types of nodes can be found here.
7. **utility.go** – Miscellaneous of scripts used by two or more different types of nodes. Scripts related to the interaction with Docker Volumes or scripts related to printing informative/debug information can be found here.
8. **sdcc.proto** – Contains the definition of every RPC Service and every RPC Message used by the system.

IV. SOLUTION DETAILS

All the project's code is in a Github repository [5]. Most of the folders, files and code calls mentioned in this section (and in the next ones) are located within this repository. Other files are automatically generated.

A. Initial Containers Setup

The first notable operation done by every container is to use a Goroutine to launch **func startServer(myname, myport string)** to set up the RPC server. Every container has its own hostname and port number. The project defines several services: one for messages related to leadership between Consensus Nodes, one for reading and writing operation and one for general messages between containers. For the sake of simplicity, every container launches a server that can manage all these types of services.

B. Client Proxy and Terminal

After the initial setup, it is possible to enter a command into the system using the terminal console managed by the Client Proxy.

When a command is inserted, the Client Proxy will first launch a Service Discovery script in which it will discover hostname and port of the new Leader Node. The Client Proxy will continue to ask to every known node until it receives an answer.

After that the Client Proxy will try to forward the command to the Leader Node. The first thing to do is check the Circuit Breaker: if it is active the entire operation stops here without

even talking to the Leader Node. Otherwise an RPC client will be created, and the operation will be forwarded to the Leader Node. The result of the operation is printed in a simplified way to the Terminal.

An operation can fail for several reasons:

- **Leader Down** – The node that Client Proxy recognizes as the Leader is currently unavailable.
- **No Leader** – No Consensus Node is in Leader State. This can happen because no Consensus Node noticed that there is no leader (more information about the Heartbeat messaging system in the next paragraph) so no one of them tried to start an election.
- **Syntax Error** – The command is provided in an incorrect syntax or with the wrong number of parameters.
- **Volume Error** – The interaction with the Docker Volume failed.

In any case - except “Syntax Error” - a failure will result in the activation of the Circuit Breaker for a fixed amount of time.

C. Leader Election and Heartbeat Messaging System

The system always starts with every Consensus Node in Follower State.

Every Consensus Node has a Struct called “HeartbeatMonitor” that contains a **time.Timer** variable.

In Raft every node starts a timer of a fixed duration called “**electionTimeout**”.

- If the node receives a Heartbeat Message during this timeframe the timer is reset again.
- If the node does not receive a Heartbeat Message and the timer reaches zero, the node will try to start an election.

When a node receives a RequestVote it will respond to the Sender in according to several factors and rules:

- If the Sender Node has an inferior Raft Term, then the Receiver Node will respond negatively without checking anything else.
- In case of equal or superior Raft Term, the Receiver Node will check the received proposal, and it will evaluate how to respond.
- Every Consensus Node can only vote once for every round of election, either for another Consensus Node or for itself.
- If the Receiver Node already voted for another Node, it will send a negative answer to the Sender Node.
- If the Receiver Node is in Candidate state, it will switch back to Follower state and it will send a positive answer to the Sender Node.
- If a Consensus Node is in Leader State and another Consensus Node tries to be elected, the former will

send a special type of negative vote that will make the election of the latter fail no matter what the other nodes voted.

When a Consensus Node is elected, it will immediately announce its election to the other Consensus Nodes.

A Leader Node has the role to periodically sends a Heartbeat Message to every Consensus Node.

D. Volume Management

Every Consensus Node has its personal Docker Volume. Every Volume contains several files, in particular:

- A file called **“myData”** that contains every pair (Key, Value) inside the system.
- A file called **“opLog”** that contains every operation that the Consensus Node did on its **“myData”** file or other notable information like the start of an election and its result.

Several functions related to the interaction with **“myData”** and **“opLog”** are defined inside **utility.go**, for example **“getValue (key string)”** or **“writeNewPair (key, value string)”**. These functions are called during the execution of the Consensus Node, in particular every time a Leader Node receives a command from the Client Proxy, or a Follower Node receives the same command from the Leader Node.

The data inside these files are formatted in a specific yet simple way and the responsibility to read and write respecting this format is assigned on the previously mentioned functions only.

E. Snapshot Node & Empirical Data Collection

This Node periodically asks to every Consensus Node the information written on their personal files and collects them.

To avoid creating another container with the specific purpose, this node is also responsible for the collection of every empirical data related to every leader election (such data are present in opLog, as mentioned before). In particular how many nodes voted and the time between the start of the election and the announcement of the winner.

Such data are collected and used to compute the mean, minimum and maximum election time, grouped by the number of nodes.

These results are periodically printed by the Snapshot Node and can be visualized in real-time by attaching to the mentioned container.

F. Testing Environment

The number of Consensus Nodes is a fixed parameter defined inside the .yaml file. For testing purposes, it was also used a different file with 15 Consensus Nodes.

Most of the tests (and related time values) were taken on local environment using that .yaml file.

Most tests also included stopping and restarting containers or stopping the Leader Node and check the reaction of every other Consensus Node.

Other tests were related to the Datastore management, in particular adding and searching pairs key value.

Lastly, tests related to the communication between the external world, Client Proxy and the Leader Node were executed to check the correctness of the Service Discovery and Circuit Breaker patterns.

V. DEPLOYMENT

The system is fully containerized with Docker and orchestrated via *docker-compose.yaml*. The project is presented in a simplified way and can be expanded in several directions. More about it is discussed in the next sections.

Core Containers:

- **clientProxyServ** – One container with the role of acting as a bridge between the outside world and the system. It communicates with the Leader Node.
- **snapshotNode** – One container with the role of reading the logs of the Consensus Nodes and compact them in snapshots useful for backup and restore purposes.
- **consensusNodeXX (XX is a number)** – A group of containers, everyone with its own name, port number and private volume. They work together to store the information in a leader-followers system coordinated by a simplified version of the Raft algorithm.

Networking & Storage

- Shared Docker bridge network: *sdcc_net*
- Private volumes *volConsXX (XX is a number)*.

A. Environment Setup

The first thing to set up is an ambient in which it is possible to run Docker. More information is provided in the GitHub repository README [5].

B. System Startup and Management

Once there is a working console connected to an environment capable of sustaining the system, the system is launched with:

```
docker compose up -d --build
```

then it is necessary to use

```
docker compose attach clientproxyserv
```

to connect the used Terminal to the system and be able to insert commands that the Client Proxy can read.

C. Container Setup and Execution

Every container is programmed to set its settings and start its main functionalities at launch using the provided Go scripts. The only recommendation to the user is to wait before starting

to write commands because the system always starts without a leader among the Consensus Nodes so every request in this timeframe will automatically fail (with consequent activation of the Circuit Breaker for a fixed amount of time).

VI. RESULTS

The system measures and stores every time-related information and Snapshot Node periodically print such information and computes statistics like the average value.

A big share of the tests in the local scenario consisted in launching the application with 15 containers, waiting for one of them to be elected as leader and then stopping such container. This operation was repeated until only one container remained.

Another kind of test was starting with six containers, waiting for one of them to become the leader and then stopping it. After the election as leader of a second container the first one was restarted so that in every election there were 5 voters.

The last kind of test simply consisted in launching the application with the default number of container (six) and simply waiting or testing other features while the Consensus Nodes started an election (and memorized the empirical data in their volumes).

Number of Nodes	Average Election Time	Min Value	Max Value	Number of Samples
1 (Local)	18.089553595s	11.843811727s	55.249636549s	7
2 (Local)	12.865552556s	7.870581456s	19.878576706s	12
3 (Local)	24.632573852s	3.986206958s	55.500933214s	14
4 (Local)	13.399947342s	8.523978ms	47.655781379s	8
5 (Local)	31.179687438s	3.960894823s	43.61978236s	8
5 (EC2 t3.small)	13.767726ms	10.564751ms	20.53058ms	10
6 (Local)	24.049819025s	14.111346ms	39.711698005s	15
6 (EC2 t3.small)	16.872582ms	11.160734ms	43.14471ms	18
12 (Local)	11.919150779s	11.884821525s	11.950306046s	5
13 (Local)	7.987334868s	7.923204297s	8.048757349s	6
14 (Local)	4.309701632s	3.982417279s	7.974785755s	13
15 (Local)	48.420731ms	30.887143ms	149.489546ms	26

Fig. 2. Collected data about election time in both local and cloud scenario.

VII. DISCUSSION

A. Number of Consensus Nodes

According to the empirical data, it seems that the system only needs an amount of time on the order of tens of milliseconds between the start of the election and the ascending to leader role of that node. Some very rare outlier appeared during the tests with the simulated crash of a leader, but it was on the order of few seconds.

It appears that trying to send an RPC message to a stopped container wastes a not-negligible amount of time so the cases with 15 total containers where most of them were stopped still wasted a big amount of time and altered the statistics. The case with one active container still required few seconds for the election despite having no proper message exchange at all.

B. Challenging Parts

Several challenges emerged during the project, especially regarding communication between nodes and technology integration.

- Learning how to configure every environment and use every software properly to create an efficient solution to the requested problem was not trivial.
- Implementing things from scratch in according to the specifics (i.e. the Raft algorithm or a key-value datastore) instead of using already existing software required a certain amount of time and produced a decent albeit simplified result.
- Managing the communication between Consensus Nodes required particular attention to several problems like deadlocks, racing condition, blocking calls and abrupt crashing of nodes. Making sure that the system could handle those problems required several tests and modifications to the code before reaching to a satisfying result.

VIII. CONCLUSIONS

The project resulted in the development of a good albeit simple key-based distributed datastore and a decent version of the Raft algorithm.

The system is portable and can be executed both locally and on Amazon AWS EC2 instance like specifics requested. A little preliminary configuration is requested in both cases, more information in the provided README file [5].

The project can be extended in several ways:

- **More replicas:** Creating two or more containers with the role of Snapshot Node or Client Proxy for an even more robust and redundant system.
- **More functionalities:** Adding new features to the datastore like the deleting of a pair or the modification of an already present key. The possibility to execute complex queries (i.e. searching for a group of keys with something in common) on the data is also a possible feature worth of mention.
- **Security issues:** Improving the strength of the system against cyber-attacks like DoS or malicious inputs.
- **Multi-user:** Expanding the Client Proxy so that the system can safely handle more users connected at the same time without consistency problems.

REFERENCES

- [1] AD. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In Proceedings of the 2014 USENIX Annual Technical Conference, USENIX ATC'14, page 305–320, 2014. <https://www.usenix.org/system/files/conference/atc14/atc14-paper-ongaro.pdf>.
- [2] Amazon Web Service (AWS) is a cloud service provided by Amazon, for more information: https://aws.amazon.com/it/what-is-aws/?nc1=f_cc
- [3] Docker Compose, "Documentation." Available online: <https://docs.docker.com/compose/>
- [4] More information about gRPC: <https://grpc.io/docs/>
- [5] AdrianoTrani, "SDCC01" [GitHub Repository], 2026. Available online: <https://github.com/AdrianoTrani/SDCC01>